Tecnologías y requisitos para trasladar y alojar Social Audit Pro

Fecha: 13 de octubre de 2026. Alcance: backend Node.js y Express, base SQLite, sincronización y PDF de una sola instancia.

Conclusión

Para pruebas en otra PC, se propone un procesador de 64 bits con 2 núcleos, 4 GB de RAM total y 10 GB libres en SSD. Para alojamiento continuo inicial, se recomienda una máquina dedicada con 2 vCPU, 4 GB de RAM asignada y 30 GB libres en almacenamiento persistente. Una PC de oficina que también ejecute navegador y otras aplicaciones debería tener al menos 8 GB de RAM total.

Estas cifras son estimaciones de planificación para pocas cuentas y uso ligero, no requisitos medidos ni garantía de usuarios simultáneos. El volumen de históricos, reportes, sincronizaciones y copias puede exigir más recursos. La prueba de carga local existente no certifica capacidad productiva.

Hardware por escenario

Recurso	PC para pruebas	Servidor dedicado de piloto permanente
CPU	2 núcleos de 64 bits	2 vCPU asignadas como punto de partida
RAM	4 GB total; preferible 8 GB con otras aplicaciones	4 GB asignados; evaluar 8 GB al crecer
Disco	10 GB libres en SSD, además del sistema operativo	30 GB libres en SSD persistente, además del sistema operativo
Red	Internet estable para instalar y consultar APIs	Conexión continua; cable Ethernet en equipo físico
Gráficos	GPU integrada suficiente	Sin GPU dedicada ni monitor requeridos
Energía	Puede apagarse al terminar las pruebas	Encendido continuo, sin suspensión; UPS recomendable en equipo físico

No existe una velocidad mínima de Internet validada. Importan disponibilidad, latencia y volumen de consultas; medir el enlace durante el piloto. Reservar espacio para código, dependencias, base, WAL, temporales, logs y copias. Las copias deben tener retención y destino externo.

Software requerido

- Sistema operativo de 64 bits mantenido, compatible con Node.js y better-sqlite3. Para PC, Windows 11; para servidor dedicado, Linux con soporte vigente. Validar la instalación en el destino.
- Node.js 22 LTS con su versión de mantenimiento actual y npm. better-sqlite3 13.0.3 instalado declara Node.js >=22. El README que indica 20 está desactualizado. Node.js 24 LTS es alternativa si pasa las pruebas del proyecto en el destino; el Dockerfile actual usa Node 22.
- Git para clonar y actualizar, o descarga ZIP del repositorio. Visual Studio Code es opcional; no es necesario para mantener el servidor encendido.
- Navegador actualizado para acceder desde una PC o móvil. El cliente no necesita Node.js ni SQLite.
- SQLite funciona embebido mediante better-sqlite3: no requiere instalar un servidor MySQL, PHP, Laravel ni XAMPP.
- Docker y Compose son opcionales. El repositorio contiene Dockerfile y compose.production.yml. Validar variables de correo, APIs y volúmenes antes de usar Compose; no asumir que replica toda configuración de .env.
- Si no hay binario nativo compatible al instalar better-sqlite3, pueden requerirse herramientas de compilación. Reinstalar dependencias en el destino en lugar de copiar node_modules entre equipos.

Red y configuración para alojamiento permanente

El servicio local escucha por defecto en 127.0.0.1:4173 y solo es accesible desde ese equipo. Un alojamiento público requiere URL estable, DNS y HTTPS mediante un proxy inverso, por ejemplo Caddy o Nginx, correctamente configurado. Exponer únicamente los accesos necesarios; no publicar el archivo de base ni .env. Mantener 4173 interno al proxy cuando corresponda.

Configurar NODE_ENV=production, APP_BASE_URL con HTTPS, DATABASE_PATH en disco persistente, HOST y PORT según el despliegue, y TRUST_PROXY solo conforme a los proxies realmente presentes. TOKEN_ENCRYPTION_KEY debe ser privado y tener al menos 32 caracteres en producción. Conservar la clave original al trasladar datos cifrados; reemplazarla arbitrariamente impide descifrar OAuth y MFA.

Para conexiones reales se necesitan aplicaciones y credenciales oficiales de cada red, permisos aprobados y callbacks OAuth idénticos a la URL registrada. Para verificación y recuperación, configurar Microsoft 365 Graph autorizado o SMTP. Permitir salida HTTPS a APIs y correo Graph, o el puerto SMTP que use el proveedor. X continúa sujeto a autorización de costes.

En una PC de oficina, publicar exige resolver acceso entrante, firewall y posibles restricciones del proveedor de Internet como CGNAT. Un dominio o una IP por sí solos no resuelven esas restricciones. El acceso por LAN también necesita configurar escucha, firewall, URL y protección adecuados. localhost siempre se refiere al dispositivo que abre el navegador.

Traslado sin perder información

1. Seleccionar un commit estable y comprobar que inicia. Existen cambios locales de autenticación pendientes de integración; no asumir que cualquier copia del directorio está lista para alojarse.
2. Preparar el sistema, Node.js 22 LTS, npm y almacenamiento del equipo destino.
3. Clonar https://github.com/TheKnight2122/social-audit-pro.git y ejecutar npm ci desde la carpeta del proyecto. No copiar node_modules.
4. Crear y verificar una copia cifrada en origen con npm run db:backup y npm run db:verify -- "backups/CARPETA-DE-LA-COPIA". Custodiar BACKUP_ENCRYPTION_KEY, TOKEN_ENCRYPTION_KEY y la configuración privada por separado. GitHub no contiene datos ni secretos del equipo.
5. Para un corte final, detener escrituras, hacer una copia final verificada y trasladar el paquete completo. No copiar directamente el archivo SQLite mientras está en uso ni ignorar su WAL.
6. Restaurar en destino con npm run db:restore -- "backups/CARPETA-DE-LA-COPIA" "data/recuperada.sqlite". El destino debe ser una ruta nueva. Configurar DATABASE_PATH a ese archivo y conservar la clave de cifrado original.
7. Ajustar .env privado, APP_BASE_URL, callbacks OAuth y correo. Una restauración invalida sesiones y enlaces pendientes y pausa tareas; iniciar sesión nuevamente y revisar integraciones antes de reactivarlas.
8. Ejecutar npm test y npm start. Comprobar /api/v1/health/ready, login, organizaciones, separación de datos, auditoría y descarga PDF. Probar redes reales solo con cuentas autorizadas. Si falla una comprobación, resolver antes del cambio definitivo.
9. Dirigir usuarios a la nueva URL y mantener la instalación anterior detenida como reserva. No operar dos copias divergentes de la base como si fueran una misma plataforma.

Operación continua

- Ejecutar como servicio del sistema o contenedor con reinicio automático y cuenta de permisos limitados. No depender de una terminal de VS Code abierta.
- Mantener disco persistente local al servidor. No alojar el archivo activo SQLite en una carpeta de red, sincronización de nube o un filesystem efímero.
- Programar copias cifradas, por ejemplo cada 24 horas para un piloto si es aceptable perder hasta un día de datos. Reducir el intervalo si el negocio lo requiere; programar copias no define por sí solo el tiempo de recuperación.
- Copiar respaldos fuera del host y probar restauración. BACKUP_INTERVAL_HOURS=0 significa copias automáticas desactivadas. La aplicación no implementa retención automática ni traslado externo de copias.
- Revisar espacio, errores, disponibilidad y vencimiento de credenciales. Monitoreo externo y alertas aún necesitan preparación productiva.
- Aplicar actualizaciones con copia previa, prueba y posibilidad de volver al código y base compatibles. Un piloto no debe prometer disponibilidad permanente ni un número de clientes sin medición.

Límites y aceptación del alojamiento

El backend activo sigue usando SQLite. Se recomienda una instancia para el piloto; la preparación parcial de PostgreSQL no habilita todavía todo el sistema en ese motor. Para varias réplicas se necesita terminar la migración, incorporar coordinación o cola compartida y probar despliegue, fallos y carga. Más RAM no completa esa arquitectura.

GitHub Pages aloja únicamente la demo estática, sin backend, autenticación persistente, sincronización ni PDF generado por el servidor. El alojamiento completo requiere un proveedor que ejecute Node.js y conserve el disco; no basta un plan que solo publique archivos estáticos.

Antes de aceptar el traslado comprobar: reinicio automático; persistencia tras reinicio; HTTPS si es público; login y aislamiento por organización; reporte PDF; correo real cuando se habilite; conector autorizado y datos reales identificados; copia externa y restauración; uso de CPU, RAM y disco con datos representativos. No se ha certificado alta disponibilidad ni capacidad productiva con este informe.

Tecnologías utilizadas en el proyecto

El sistema separa interfaz web, analítica, API REST, autenticación, persistencia, conectores y tareas de sincronización. La siguiente tabla distingue las tecnologías activas de los componentes preparados o pendientes de despliegue.

Componente	Tecnología	Uso y estado
Interfaz y gráficos	HTML, CSS y JavaScript	Vistas, filtros, cálculos y gráficos propios. Sin React, Angular ni Chart.js.
Backend	Node.js y Express	API REST bajo /api/v1, permisos y ejecución del servidor local.
Base de datos activa	SQLite y better-sqlite3	Archivo persistente local para usuarios, sesiones, organizaciones, publicaciones, históricos y reportes.
Reportes	PDFKit	Generación de documentos PDF desde el backend.
Correo	Nodemailer y Microsoft Graph	SMTP disponible; transporte Microsoft 365 preparado, sujeto a autorización y configuración real.
Segundo factor	otplib y qrcode	Códigos TOTP y QR para configurar autenticadores.
APIs de redes	googleapis y adaptadores OAuth	YouTube, Facebook, Instagram, TikTok, LinkedIn y X. Su implementación no confirma conexión real; necesitan permisos, credenciales y validación. X está bloqueado por presupuesto.
Configuración	dotenv	Lectura de variables privadas del entorno. Los secretos no se guardan en GitHub.
Pruebas	node:test y Supertest	Pruebas unitarias y HTTP. PGlite se usa en pruebas de adaptadores PostgreSQL.
Versionado y publicación	Git, GitHub y GitHub Actions	Historial del código y flujo de publicación de la demostración.
Hosting público actual	GitHub Pages	Solo frontend estático de demostración. No aloja el backend ni SQLite.
Despliegue preparado	Docker y Docker Compose	Archivos para una instancia con volúmenes persistentes. No acreditan un despliegue productivo activo.
Migración parcial	PostgreSQL y pg	Esquema y adaptadores preparados. El backend completo sigue usando SQLite.

Visual Studio Code permite editar y ejecutar comandos, pero no es una dependencia del servidor. No se utiliza PHP, Laravel, MySQL, XAMPP ni Hostinger en el alojamiento actual del proyecto. El hardware del informe corresponde al backend completo de una instancia; para visitar la demostración pública basta un navegador.

Referencias

- Repositorio: https://github.com/TheKnight2122/social-audit-pro
- Demo: https://theknight2122.github.io/social-audit-pro/
- Evidencia local revisada: package.json, node_modules/better-sqlite3/package.json, server.js, .env.example, Dockerfile, compose.production.yml, docs/28-produccion-y-alta-disponibilidad.md, docs/29-copias-y-restauracion.md y docs/31-validacion-concurrencia-y-carga.md.
- Node.js y versiones LTS: https://nodejs.org/en/about/previous-releases
- Usos y límites de SQLite: https://sqlite.org/whentouse.html
- Naturaleza estática de GitHub Pages: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages